
用户信息管理平台

安全漏洞挖掘与修复报告

<http://10.133.25.191:5000>

项目版本：V6.0 — 权限提升 + 业务逻辑漏洞专题

报告日期：2026年7月10日

今日课程：Day4:权限提升与业务逻辑漏洞

技术栈：Python Flask / IDOR / 余额操作 / SQL注入

一、漏洞分析

在已有功能基础上新增了个人中心和充值功能，由于未做权限校验和业务逻辑检查，存在2个严重漏洞。

1.1 新增功能

功能	路由	说明
个人中心	/profile?user_id=X	查看用户资料，user_id来源于URL参数
充值	/recharge	修改余额，amount来源于表单参数

1.2 漏洞1：IDOR（权限提升）

风险等级：高危

个人中心从 URL 参数获取 user_id，但未验证当前登录用户与查询的 user_id 是否匹配。

```
// 漏洞代码
user_id = request.args.get("user_id")
user = _get_user_by_id(user_id) // 直接查询，不验证身份

// 攻击方法：修改URL参数
/profile?user_id=1 // 查看admin的资料
/profile?user_id=2 // 查看alice的资料（越权！）
/profile?user_id=3 // 查看任意注册用户的资料
```

1.3 漏洞2：业务逻辑漏洞（金额为负）

风险等级：高危

充值接口直接拼接 SQL 更新余额，但未校验 amount 的正负。攻击者可填写负数实现盗刷。

```
// 漏洞代码
sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
// 没有检查 amount 是否 > 0

// 攻击方法：提交负值
POST /recharge
user_id=1&amount=-100000 // 余额减少10万！
user_id=2&amount=999999 // 给alice加钱
```

二、修复方案

2.1 修复IDOR漏洞

从 session 获取当前用户，不允许查询其他用户资料：

```
// 修复后：从session获取当前用户
@app.route("/profile")
@login_required
def profile():
    username = session.get("username")
    user = _get_user(username) // 只能查自己
    return render_template("profile.html", user=user)
```

2.2 修复业务逻辑漏洞

检查金额必须为正数，使用参数化查询：

```
// 修复后：校验金额正负
amount = int(request.form.get("amount", "0"))
if amount <= 0:
    return "金额必须为正数"

// 使用参数化查询
c.execute("UPDATE users SET balance = balance + ? WHERE username = ?",
          (amount, session["username"]))
```

三、POC验证结果

POC	测试方法	修复前结果	修复后
IDOR	登录admin后访问/profile?user_id=2	查到alice的资料 ✓	只能查自己
负值充值	POST amount=-100000	余额减少10万 ✓	拦截负值

服务端日志记录

```
[业务逻辑漏洞] 执行SQL: UPDATE users SET balance = balance + 500 WHERE id = 1
[业务逻辑漏洞] 用户ID 1 余额变动: +500
[业务逻辑漏洞] 执行SQL: UPDATE users SET balance = balance + -100000 WHERE id = 1
[业务逻辑漏洞] 用户ID 1 余额变动: -100000
```

四、总结

漏洞类型	根因	修复方案
IDOR（权限提升）	未验证user_id归属	从session获取用户身份，而非URL参数
业务逻辑漏洞	未校验金额正负 + 拼接SQL	金额>0校验 + 参数化查询

核心安全原则：

- 1. 永远不要信任前端传入的用户标识（user_id应从session获取）
- 2. 所有业务操作必须做权限校验（用户只能操作自己的数据）
- 3. 数值型字段必须校验范围（金额不能为负）
- 4. 所有SQL必须使用参数化查询

— 报告完 —